

Lead Engineer Take-Home Test — Klontong Admin

Stack: Nuxt 3 + NestJS

Latar Belakang

Sebuah toko klontong ingin memodernisasi operasionalnya dan mengelola produk untuk penjualan online. Saat ini tooling yang ada masih sangat sederhana. Kami ingin melihat bagaimana Anda merancang dan membangun versi yang lebih maintainable, mudah dikembangkan, dan tetap sederhana untuk digunakan oleh staf toko yang non-teknis.

Ini adalah take-home test untuk **level lead programmer / lead full-stack engineer**. Kami tidak hanya menilai apakah aplikasi berjalan, tetapi juga bagaimana Anda membuat keputusan teknis, menyusun arsitektur, menjaga kualitas kode, dan mengomunikasikan trade-off dengan jelas.

Ekspektasi Waktu

Silakan kerjakan assignment ini dalam waktu **maksimal 8 jam**.

Jika tidak sempat menyelesaikan semua bagian:

- prioritaskan vertical slice yang paling penting,
- jelaskan trade-off yang Anda ambil,
- dan tuliskan apa yang akan Anda lanjutkan jika memiliki waktu tambahan.

Kami lebih menghargai solusi yang fokus, rapi, dan well-reasoned dibanding solusi besar tetapi kurang terstruktur.

Stack yang Wajib Digunakan

- Frontend: **Nuxt 3**
- State management: **Pinia**
- Backend: **NestJS**
- Database: **PostgreSQL** atau **MongoDB**

Anda bebas menggunakan library tambahan apa pun yang menurut Anda relevan untuk production-grade application.

Ekspektasi Teknis

- Frontend dan backend harus dibuat sebagai **aplikasi terpisah**.
 - Semua request dan response harus menggunakan **JSON**.
 - Backend harus menyediakan HTTP API yang rapi dan konsisten.
 - Backend harus memiliki **validation** untuk input payload.
 - Frontend harus mengonsumsi API melalui struktur yang jelas, misalnya service layer / composables.
 - Struktur project, naming, dan separation of concerns harus jelas.
-

Brief Produk

Bangun sebuah admin system sederhana untuk mengelola produk toko klontong.

Contoh skema produk:

```
{
  "id": 86,
  "categoryId": 14,
  "categoryName": "Snacks",
  "sku": "MHZVTK",
  "name": "Ciki Ciki",
  "description": "A popular snack sold by our store",
  "weight": 500,
  "width": 5,
  "length": 5,
  "height": 5,
  "image": "https://example.com/image.jpg",
  "price": 30000,
  "isActive": true,
  "stock": 120
}
```

Silakan Anda sesuaikan field jika menurut Anda ada model yang lebih baik, selama alasannya dijelaskan.

Fitur Wajib

1. Authentication

Sediakan:

- Register
- Login
- Protected pages untuk product management

Anda bebas memilih pendekatan authentication (misalnya session-based atau token-based), tetapi mohon jelaskan alasannya secara singkat di README atau architecture note.

2. Product Management

Sediakan fitur:

- List product
 - Product detail
 - Create product
 - Edit product
 - Soft delete atau archive product
-

3. Product List Experience

Halaman list product wajib memiliki:

- Server-side pagination
- Search berdasarkan nama produk atau SKU

- Filter berdasarkan kategori atau status aktif
 - Sorting minimal pada satu field
 - Loading state
 - Empty state
 - Error state
-

4. Data Model dan Validation

Wajib ada:

- Schema/model yang jelas
 - Validation di backend
 - Constraint yang masuk akal, misalnya SKU harus unique
 - Seeder untuk sekitar **100 produk**
-

Ekspektasi Khusus untuk Level Lead

Selain aplikasi berjalan, kami juga ingin melihat kualitas engineering leadership Anda.

A. Architecture Note

Sertakan dokumen singkat sekitar **1–2 halaman** yang menjelaskan:

- Mengapa Anda memilih PostgreSQL atau MongoDB
- Bagaimana Anda menyusun struktur backend di NestJS
- Bagaimana Anda menyusun struktur frontend di Nuxt 3
- Bagaimana API contract Anda dirancang
- Trade-off apa yang Anda ambil karena batas waktu
- Bagaimana sistem ini akan dikembangkan dari 100 produk menjadi 100.000 produk
- Bagaimana Anda akan menambahkan role-based access control
- Bagaimana Anda akan menambahkan audit logging
- Apa yang sengaja tidak Anda build dan mengapa

Yang kami cari di sini adalah cara berpikir Anda, bukan panjang dokumennya.

B. Production Gaps

Tambahkan section di README dengan judul **Production Gaps** yang membahas:

- Security concerns
- Performance concerns
- Logging / observability
- Deployment considerations
- Testing gaps
- Data migration concerns

Kami ingin melihat apakah Anda mampu membedakan solusi “cukup untuk assignment” vs “siap production”.

C. Team Handoff Quality

README wajib berisi:

- Langkah setup project
- Daftar environment variables
- Cara menjalankan migration / schema setup
- Cara menjalankan seeder
- Cara menjalankan test
- Akun user yang bisa dipakai untuk testing
- Gambaran struktur folder/project
- Penjelasan singkat keputusan arsitektur utama

Anggap README Anda akan dibaca oleh dua engineer lain yang harus melanjutkan project ini tanpa bantuan Anda.

Nice to Have

Bagian ini **opsional**, tetapi akan menjadi nilai tambah jika dikerjakan dengan baik:

- Responsive mobile-first UI
- Search state persistence
- Debounced search
- Docker / docker-compose
- Basic CI workflow
- Role support (misalnya admin dan staff)
- Swagger / OpenAPI documentation
- Image fallback handling
- Optimistic UI update

Jangan mengerjakan semuanya jika itu membuat kualitas solusi utama turun. Pilih yang paling meaningful.

Deliverables

Silakan submit repository atau monorepo yang berisi:

- Frontend app (Nuxt 3)
- Backend app (NestJS)
- README
- Architecture note
- .env.example
- Schema / migrations
- Seeder
- Sample test account credentials

Contoh struktur folder yang kami harapkan:

```
/apps  
  /frontend  
  /backend  
README.md  
.env.example
```

Struktur yang berbeda juga boleh, selama tetap jelas dan mudah dijalankan.